

Table of Contents

TABLE OF CONTENTS	1
K-MEANS CLUSTERING	3
Introduction to Unsupervised Learning and Clustering.....	4
Supervised vs Unsupervised Learning.....	4
K-Means Clustering.....	5
Example	5
Why Is K-Means Required?	6
Real-Life Analogy: Grouping Students	7
Why Is It Called K-Means?	7
Meaning of “K”	8
Meaning of “Means”	8
When Should K-Means Be Used?	8
When Should K-Means Not Be Used?	9
Important K-Means Terminology	9
Distance and Similarity Measure	10
Euclidean Distance	11
Squared Euclidean Distance.....	11
Why Does K-Means Use Squared Distance?	12
Example	12
Centroids or Cluster Centers	12
Why Is a Centroid Required?	13
Centroid Formula	13

Numerical Example: Calculating a Centroid	13
Important Properties of a Centroid.....	14
Limitation of Centroids	15
Good vs Bad Clusters	16
WCSS and Inertia.....	17
What Is Inertia?	17
Numerical Example: Calculating WCSS.....	18
Working Process	18
Practical Considerations	18
Features on Different Scales	19
Choice of K.....	19
Influence of Outliers.....	20
Initial Centroid Values	20
Stopping Criteria	21
Dimensionality	22
Presence of Categorical Variables.....	22
Other Forms of Clustering	23
Applications of Clustering	24
Python Implementation of K-Means Clustering.....	24
Business Problem	25
HIERARCHICAL CLUSTERING.....	31
Why Is Hierarchical Clustering Required?	31
Real-Life Analogy: Organizing Books	32
TYPES OF HIERARCHICAL CLUSTERING.....	32
Agglomerative Hierarchical Clustering.....	32
Working Process	33
Customer-Segmentation Example	33
DIVISIVE HIERARCHICAL CLUSTERING	34
Agglomerative vs Divisive Clustering :-	35
DENDROGRAM	36
Linkage Methods in Hierarchical Clustering	36
Advantages of Hierarchical Clustering	37
Limitations of Hierarchical Clustering.....	37

FUZZY CLUSTERING	37
Why Is Fuzzy Clustering Required?	38
Real-Life Example of Fuzzy Clustering	38
Advantages.....	39
Disadvantages.....	39
Limitations of Fuzzy Clustering	39
Example	40
Limitation	40

K-Means Clustering

Introduction to Unsupervised Learning and Clustering

Unsupervised learning is a way for machines to discover hidden patterns in data without any labels or guidance. Instead of being told what to look for, the model explores the data on its own to find structure and relationships that naturally exist. This makes it especially useful when there are no clear categories or outcomes available, but we still want to understand how the data is organised or what insights it might hold.

In simple terms, it's like letting someone wander through a new city without a map. They start noticing which areas look alike, which shops attract similar crowds, and where distinct regions begin to form. Similarly, a computer analyses data to spot similarities, differences, and trends that humans might overlook, especially when the dataset is large or complex.

Clustering is one of the most common and important forms of unsupervised learning. Its main goal is to group similar data points together without any labels or predefined categories. By uncovering natural patterns and relationships within data, clustering helps reveal structure that isn't immediately visible and makes it easier to interpret complex datasets. It is widely used for exploring data, identifying segments, and discovering meaningful patterns across fields such as marketing, healthcare, and security.

While clustering is the most familiar example, unsupervised learning also includes other methods such as dimensionality reduction and association learning. These techniques further support data exploration by simplifying information or finding relationships between items. Together, they help reveal the hidden organisation within unlabelled data and guide deeper understanding.

Supervised vs Unsupervised Learning

Given below are the general differences between supervised and unsupervised learning.

Aspect	Supervised Learning	Unsupervised Learning
Data Type	Uses labelled data where inputs	Uses unlabelled data with no

	have known outputs	predefined answers
Goal	Learns to predict outcomes or classify new data based on past examples	Finds hidden patterns, structures, or groupings within data
Examples of Use	Spam detection, price prediction, sentiment analysis	Customer segmentation, topic modelling, anomaly detection
Feedback and Accuracy	Model performance can be measured against known results	No direct measure of accuracy since true labels are unknown

K-Means Clustering

K-means clustering is one of the simplest and most popular techniques used to group data in unsupervised learning. The idea is to divide data points into a set number of clusters, where each cluster contains points that are more similar to each other than to those in other clusters.

The algorithm works by repeatedly assigning points to the nearest cluster centre (called a *centroid*) and then updating these centres based on the average position of the points within each cluster. This process continues until the groupings no longer change significantly.

K-means is widely used because it is fast, easy to understand, and effective for finding clear patterns or natural divisions in data, such as grouping customers by behaviour or segmenting images by colour.

To understand how K-means actually forms these groups, we need to look at the key ideas it relies on, such as measures of distance, the concept of similarity, and the role of centroids in defining each cluster.

Example

Suppose an e-commerce company has information about 10,000 customers.

The available features are:

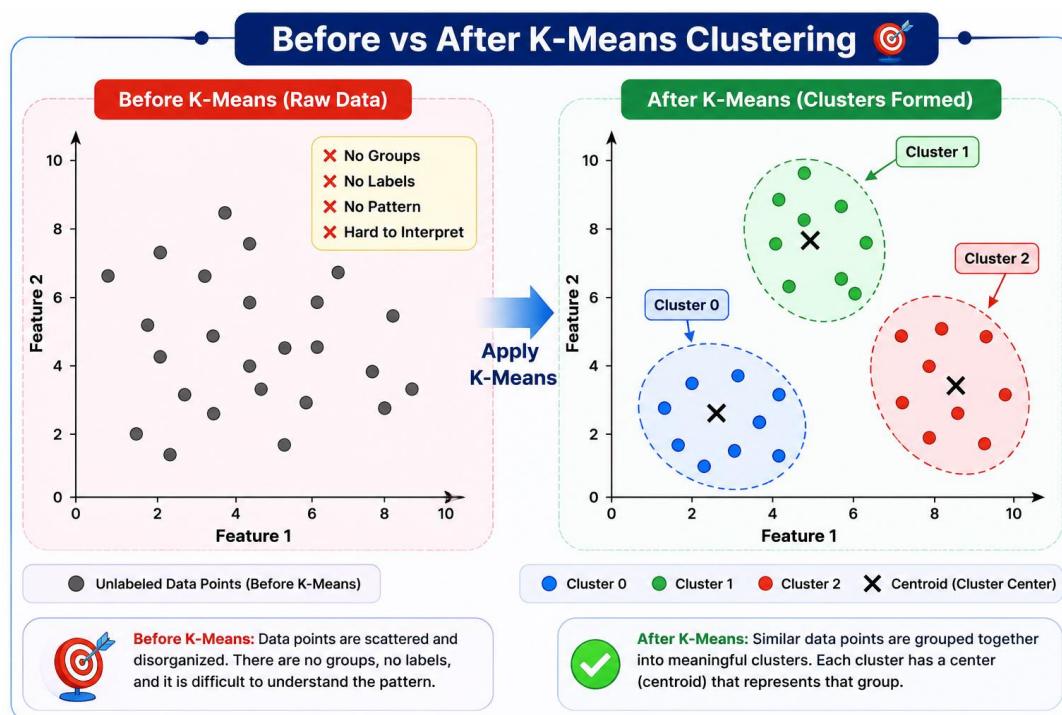
Customer	Annual Income	Annual Spending	Purchase Frequency
----------	---------------	-----------------	--------------------

CustomerA	₹3,00,000	₹40,000	5
CustomerB	₹3,50,000	₹45,000	6
CustomerC	₹12,00,000	₹5,00,000	40
CustomerD	₹11,00,000	₹4,50,000	35

The company does not have predefined customer categories. Therefore, it cannot directly use supervised classification because no target labels are available.

K-Means may analyse the customer characteristics and discover groups such as:

Cluster	Business Interpretation
Cluster 0	Budget Customers
Cluster 1	Premium Customers
Cluster 2	Potential Customers



Why Is K-Means Required?

Many real-world datasets do not contain target labels.

For example, a company may have customer information such as:

- Age
- Income

- Total spending
- Number of purchases
- Website activity
- Product preferences

However, the company may not know the customer segments in advance.

K-Means helps discover natural groups hidden within the data. These groups can then support business decisions such as personalised marketing, customer retention, product recommendations, and pricing strategies.

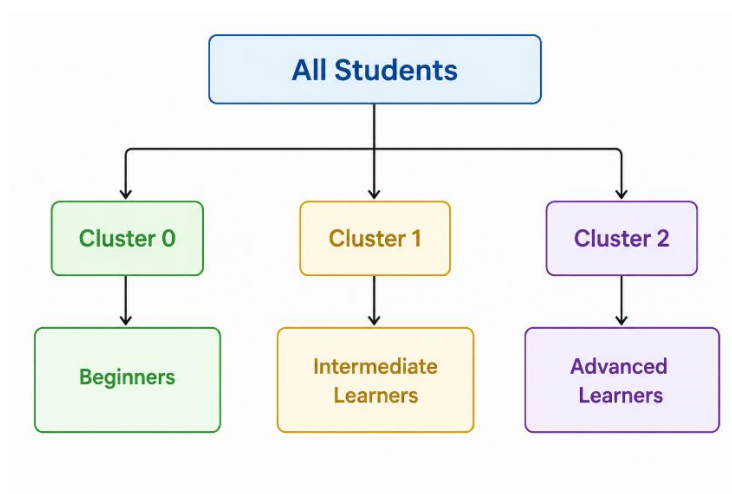
Real-Life Analogy: Grouping Students

Imagine a teacher wants to divide students into three learning groups based on:

- Mathematics score
- Programming score

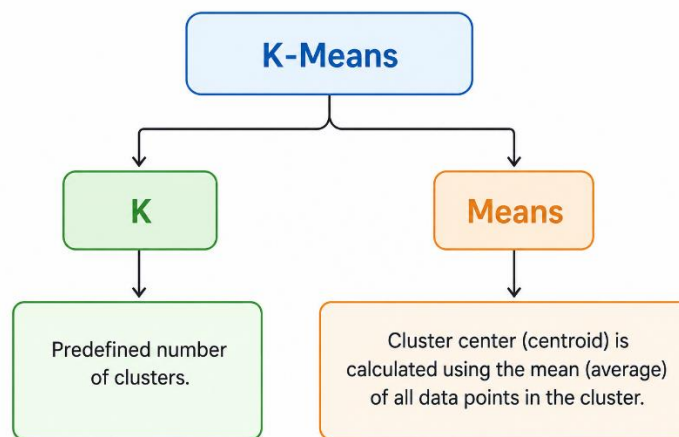
The teacher does not already know which student belongs to which group.

After analysing the scores, K-Means may discover:



Why Is It Called K-Means?

The name **K-Means** consists of two parts:



Meaning of “K”

The letter **K** represents the number of clusters that the algorithm should create.

Examples:

Value of K	Number of Clusters
K = 2	2 clusters
K = 3	3 clusters

Meaning of “Means”

The word **Means** refers to the arithmetic mean used to calculate the center of every cluster.

Suppose one cluster contains the following one-dimensional observations:

10, 20, 30

The centroid is:

$$\text{Centroid} = (10 + 20 + 30)/3 = 20$$

Therefore, the center of the cluster is 20.

When Should K-Means Be Used?

K-Means is generally suitable when:

- The dataset is primarily numerical.
- Target labels are unavailable.
- The number of clusters can be estimated or evaluated.
- Distance is a meaningful measure of similarity.
- Clusters are expected to be reasonably compact.
- The dataset is large and computational efficiency is important.
- Business users require clear, interpretable segments.

When Should K-Means Not Be Used?

K-Means may not be appropriate when:

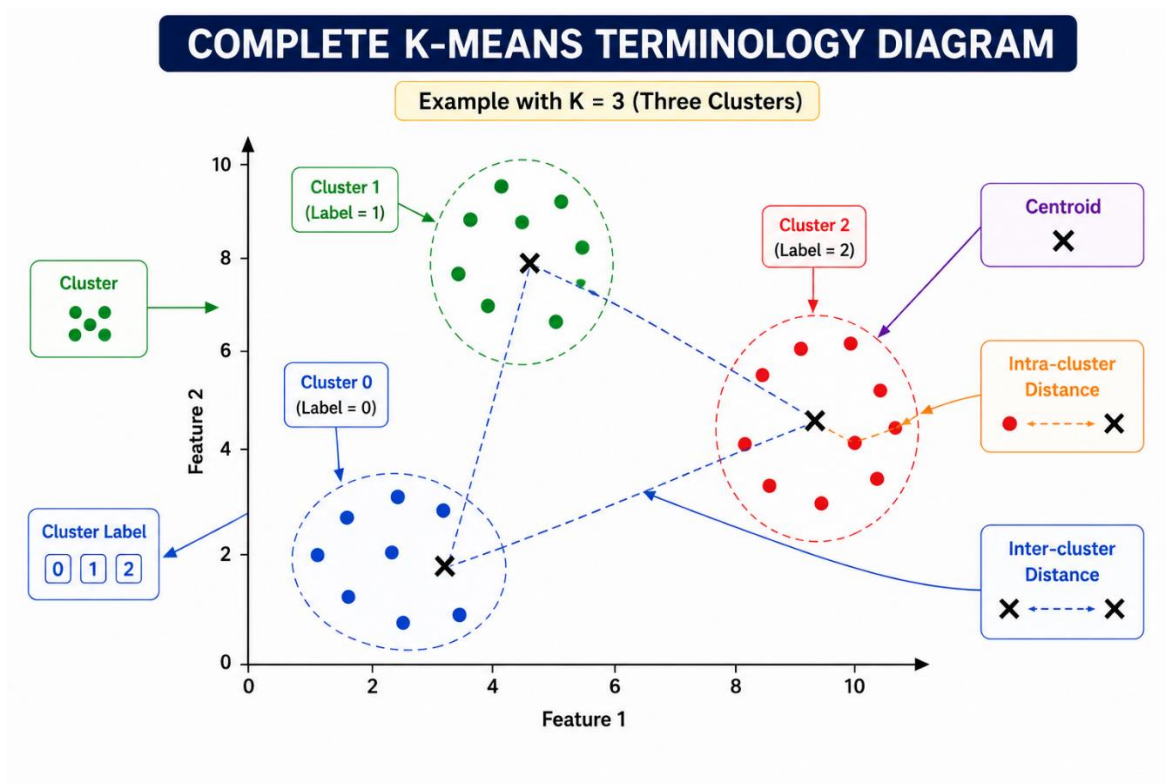
- The dataset mainly contains categorical variables.
- The data contains many severe outliers.
- Clusters have highly irregular or non-convex shapes.
- Cluster densities are extremely different.
- Cluster sizes are strongly imbalanced.
- The correct number of clusters cannot be meaningfully estimated.
- Euclidean distance does not represent similarity appropriately

Important K-Means Terminology

Before studying the mathematical working of K-Means, students should understand the following terms:

Term	Meaning
Cluster	A group of similar observations
Centroid	The mean position of a cluster

Cluster label	A numerical identifier assigned to a cluster
K	The predefined number of clusters
Intra-cluster distance	Distance among observations within the same cluster
Inter-cluster distance	Distance between different clusters



Distance and Similarity Measure

Distance measures how far two data points are from each other. In K-Means clustering, distance is used to determine which data points are similar and which centroid is closest to each data point.

K-Means generally follows this idea:

Small Distance → High Similarity

Large Distance → Low Similarity

Suppose three customers have the following annual income and spending scores:

Customer	Annual Income	Spending Score
----------	---------------	----------------

A	20	30
B	22	32
C	80	90

Customers A and B have similar values. Therefore, the distance between them is small.

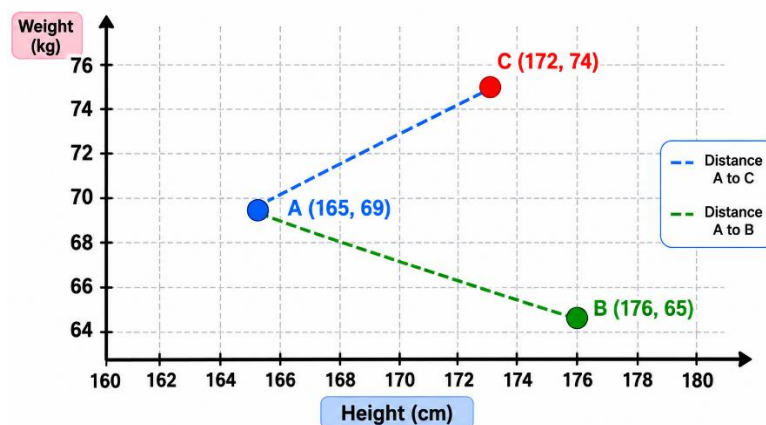
Customer C has values that are very different from A and B. Therefore, Customer C is farther away and may belong to another cluster

Euclidean Distance

The Euclidean distance between two points can be found by applying the formula $d(x, y) = \sqrt{(\sum_{i=1}^n (x_i - y_i)^2)}$. Here, the terms x_i and y_i represent the coordinates (or feature values) of the two points being compared.

For instance, consider the following as a depiction of how distance can be calculated.

Person	Height (cm)	Weight (kg)	Coordinate (x, y)
A	165	69	(165, 69)
B	176	65	(176, 65)
C	172	74	(172, 74)



Distance Between A and B

Coordinates

- $A = (165, 69)$
- $B = (176, 65)$

Step 1

$$\begin{aligned}d_{AB} &= \sqrt{(176 - 165)^2 + (65 - 69)^2} \\&= \sqrt{11^2 + (-4)^2} \\&= \sqrt{121 + 16} \\&= \sqrt{137} \\&\boxed{d_{AB} \approx 11.7}\end{aligned}$$

2. Distance Between A and C

Coordinates

- $A = (165, 69)$
- $C = (172, 74)$

Step 1

$$\begin{aligned}d_{AC} &= \sqrt{(172 - 165)^2 + (74 - 69)^2} \\&= \sqrt{7^2 + 5^2} \\&= \sqrt{49 + 25} \\&= \sqrt{74} \\&\boxed{d_{AC} \approx 8.6}\end{aligned}$$

3. Compare the Distances

Distance	Value
d_{AB}	11.7
d_{AC}	8.6

$$8.6 < 11.7$$

Conclusion:

Person C is closer to Person A than Person B.

Person A is more similar to Person C based on height and weight.

Squared Euclidean Distance

Squared Euclidean distance is calculated by adding the squared differences between corresponding feature values without calculating the final square root.

For the previous example:

Point A = (2, 3)

Point B = (5, 7)

The squared distance is:

$$\begin{aligned}
 &(5 - 2)^2 + (7 - 3)^2 \\
 &= 3^2 + 4^2 \\
 &= 9 + 16 \\
 &= 25
 \end{aligned}$$

$$\text{Squared Euclidean Distance} = 25$$

Why Does K-Means Use Squared Distance?

K-Means uses squared distance because:

- It gives a larger penalty to points that are far from their centroid.
- Squared values are always non-negative.
- It works naturally with the arithmetic mean.
- The square-root calculation is unnecessary when comparing distances.
- It makes mathematical optimisation easier.

Example

Original Distance	Squared Distance
1	1
2	4
5	25
10	100

Centroids or Cluster Centers

A **centroid** or **cluster centre** represents the average position of all points within a cluster. It is calculated by taking the mean value of each feature for the points assigned to that cluster, giving a single representative point in the data space.

During clustering, each data point is assigned to the nearest centroid using a distance measure, often the Euclidean distance. The centroids are then updated to reflect the new cluster assignments, and this process repeats until the centroids no longer change significantly.

Centroids help describe the main characteristics of each cluster and make it easier to compare groups within a dataset. However, since they are based on averages, they may not fully capture the variation or irregular shapes that exist in real data.

For example, if customers are represented using two features—**annual income** and **annual spending**—the centroid represents the average income and average spending of all customers assigned to that cluster.

Why is a Centroid Required?

K-Means requires a reference point to determine which data points should belong to each cluster. The centroid serves as this reference point.

During clustering, the algorithm calculates the distance between every data point and every centroid. Each data point is then assigned to the cluster containing its nearest centroid.

The centroid performs three important functions:

1. It represents the average position of a cluster.
2. It helps determine the cluster membership of each data point.
3. It is repeatedly updated as cluster assignments change.

Without centroids, K-Means would not have a central reference for comparing and grouping data points.

Centroid Formula

Below the image, keep the explanation as editable text:

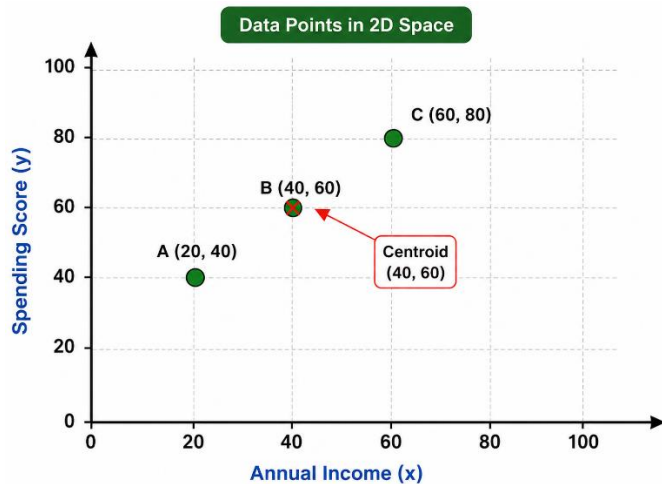
- μ_j = Centroid of cluster j
- C_j = Set of all data points belonging to cluster j
- $|C_j|$ = Number of data points in cluster j
- x_i = An individual data point belonging to cluster j

Numerical Example: Calculating a Centroid

Suppose one cluster contains the following customers:

Customer	Annual Income (x)	Spending Score (y)
A	20	40

B	40	60
C	60	80



Calculate Mean of Annual Income (x)

$$\begin{aligned}
 \text{Mean Income} &= \frac{20 + 40 + 60}{3} \\
 &= \frac{120}{3} \\
 &= 40
 \end{aligned}$$

Mean Annual Income = 40

Calculate Mean of Spending Score (y)

$$\begin{aligned}
 \text{Mean Spending Score} &= \frac{40 + 60 + 80}{3} \\
 &= \frac{180}{3} = 60
 \end{aligned}$$

Mean Spending Score = 60

Calculate the Centroid

The centroid is given by:

$$\text{Centroid} = (\text{Mean of } x, \text{Mean of } y)$$

Substitute the calculated means:

$$= (40, 60)$$

Final Answer

Centroid = (40, 60)

Important Properties of a Centroid

- Every cluster has one centroid.
- A centroid is calculated using the arithmetic mean.
- A centroid contains one mean value for every feature.
- A centroid may not be an actual observation in the dataset.
- Centroid positions may change during model training.
- Data points are assigned according to their distance from the centroids.
- Centroids are sensitive to extreme values because they are based on arithmetic means.

Limitation of Centroids

Centroids are based on arithmetic means and may not completely represent the internal variation of a cluster.

For example, two clusters may have similar centroid positions but very different distributions:

- One cluster may be compact and tightly grouped.
- Another cluster may be widely dispersed.

Centroids may also provide a poor representation when clusters have:

- Irregular shapes
- Strongly unequal densities
- Large internal variation
- Severe outliers

Therefore, centroids should be interpreted together with cluster evaluation measures such as:

- WCSS or inertia

- Silhouette score
- Cluster stability
- Business interpretability

Important Note: Because centroids are calculated using arithmetic means, extreme values can pull centroids away from the majority of data points. Therefore, outliers should be investigated before applying K-Means.

Good vs Bad Clusters

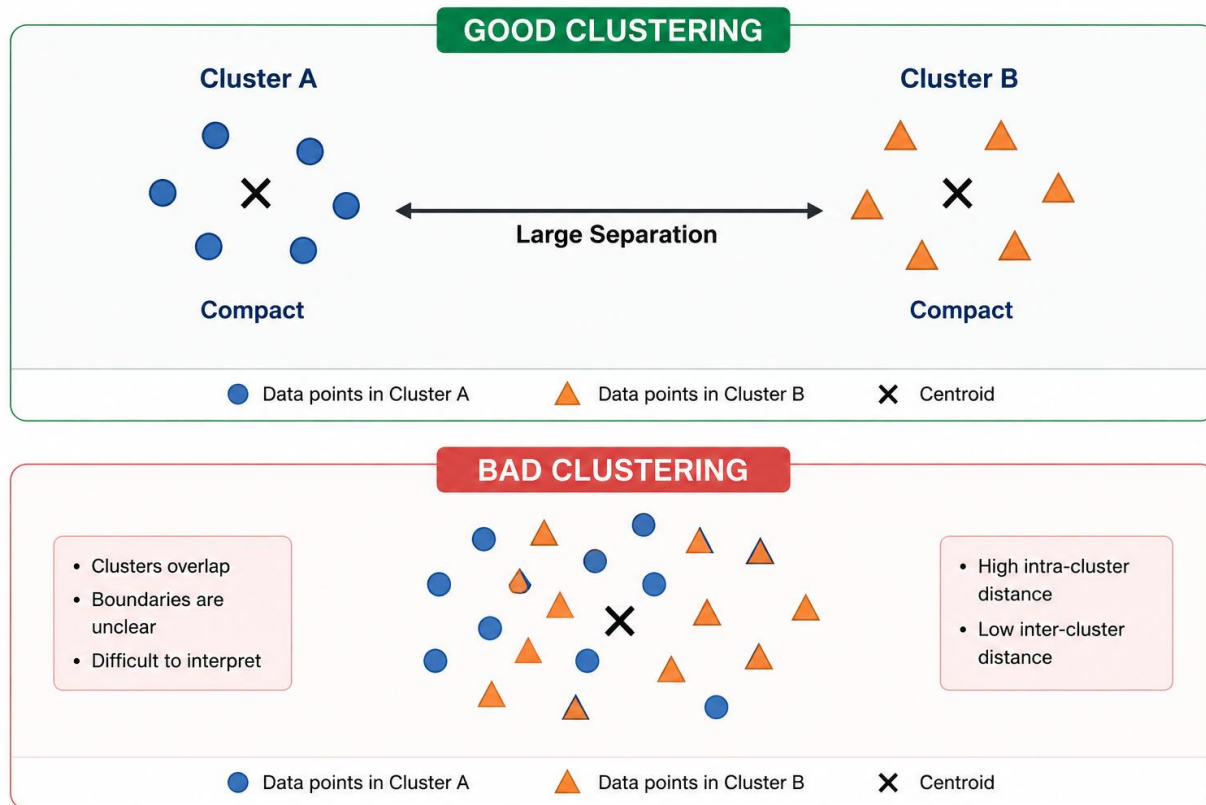
A cluster may be said to be good if:

- Internal (intra-cluster or within the cluster) distances should be small
 - This can be captured by using the within-cluster sum of squares (WCSS) or inertia which is given by the formula $WCSS = \sum_{i=1}^n \|x_i - C\|^2$ where C is the centroid
- External (inter-cluster or between-cluster) distances should be large

Refer to the following table to understand the difference between good and bad clusters.

Aspect	Good Clustering	Bad Clustering
Compactness	Low WCSS, meaning points are tightly grouped around the centroid	High WCSS, indicating large variation within clusters
Separation	High distances between clusters, showing clusters are well-separated	Low distances between clusters, suggesting that clusters are not distinct from one another
Silhouette Score	High (close to 1), meaning points are well matched to their own cluster	Low or negative, suggesting poor or overlapping clustering
Cluster Shape	Compact and roughly spherical (for distance-based methods like K-Means)	Irregular or elongated, violating algorithm assumptions

Interpretability	Clusters make sense in the problem context	Clusters are hard to explain or lack real meaning
-------------------------	--	---



WCSS and Inertia

WCSS stands for **Within-Cluster Sum of Squares**. It measures the total squared distance between every data point and the centroid of the cluster to which it is assigned.

WCSS measures the **compactness** or **within-cluster variation** of a clustering solution. When data points are located close to their assigned centroids, the WCSS value is relatively low. When data points are widely spread and far from their centroids, the WCSS value is high.

The main objective of K-Means is to minimise WCSS so that the data points within each cluster remain as close as possible to their cluster centroid.

What Is Inertia?

Inertia is the term used by Scikit-learn for the final WCSS value calculated by a trained K-Means model.

Therefore, in Scikit-learn's standard K-Means implementation:

WCSS = Inertia

Both represent:

The total sum of the squared distances between every data point and its assigned cluster centroid.

Numerical Example: Calculating WCSS

Cluster 1

Data Points: 2, 4, 6

Centroid (μ_1) = 4

Data Point (x_i)	Distance ($x_i - \mu_1$)	Squared Distance ($(x_i - \mu_1)^2$)
2	$2 - 4 = -2$	4
4	$4 - 4 = 0$	0
6	$6 - 4 = 2$	4

$$\text{WCSS of Cluster 1} = 4 + 0 + 4 = 8$$

$$\begin{aligned} \text{Total WCSS} &= \text{WCSS}_{\text{Cluster 1}} + \text{WCSS}_{\text{Cluster 2}} \\ &= 8 + 8 = 16 \end{aligned}$$

$$\text{Total WCSS} = 16$$

Working Process

The K-means clustering algorithm works as follows

- Choose the **number of clusters** (K) to form

- **Randomly initialise** K centroids from the dataset
- **Assign** each data point to the **nearest centroid** based on distance
- **Recalculate centroids** as the mean of assigned points
- Repeat assignment and update steps until centroids **stabilise**
- Evaluate clustering quality using WCSS or silhouette score

Features on Different Scales

K-means clustering is highly sensitive to the scale of features because it uses distance-based calculations, typically the Euclidean distance. When features have different ranges or units, those with larger values can dominate the distance computation, leading to biased clustering results.

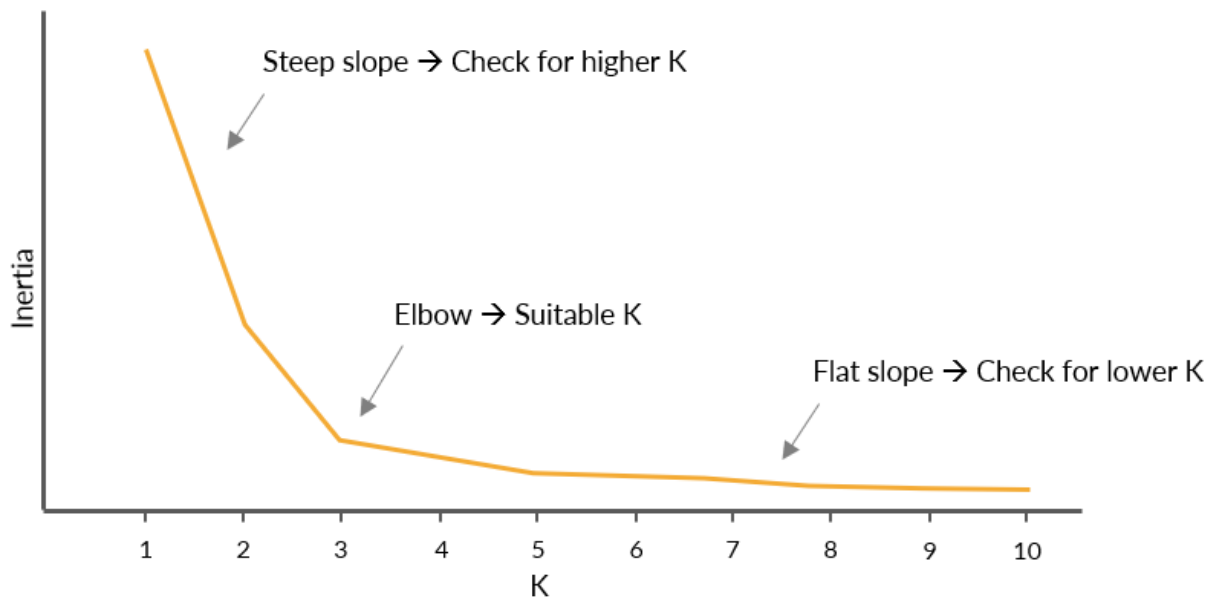
For example, in a customer segmentation task, annual income might be measured in lakhs of rupees, while customer satisfaction rating is on a scale of 1 to 5. Without scaling, the large numeric range of income would dominate the clustering process, causing the algorithm to form clusters mainly based on income and largely ignore satisfaction levels, even if those ratings are crucial for meaningful insights.

To avoid this imbalance, it is essential to scale features before applying K-means. Standardisation (converting values to have zero mean and unit variance) or normalisation (scaling values to a fixed range, such as 0 to 1) ensures that all features contribute equally. This step helps produce clusters that genuinely represent the structure of the data rather than the influence of measurement scales.

Choice of K

Choosing the right number of clusters, K , is one of the most important decisions in K-means clustering. The **elbow method** is a common approach for this. It involves plotting the Within-Cluster Sum of Squares (WCSS) against different values of K and looking for a point where the rate of decrease sharply slows down — the “elbow.” This point suggests a balance between compact clusters and model simplicity.

Consider the following plot of inertia versus K . The “elbow” being formed here suggests an optimal value of K .



Another useful technique is the **silhouette score**, which measures how similar each point is to its own cluster compared to other clusters. The score ranges from -1 to 1 , with higher values indicating better-defined clusters. By comparing silhouette scores across different K values, one can identify the configuration that provides the most cohesive and well-separated clustering.

Note that, in practical applications, the ideal K may not always align with what these methods suggest. **Business domain knowledge** often plays a key role. For instance, a marketing team might prefer three customer segments because they match existing strategies, even if the data-driven optimum is slightly different.

Influence of Outliers

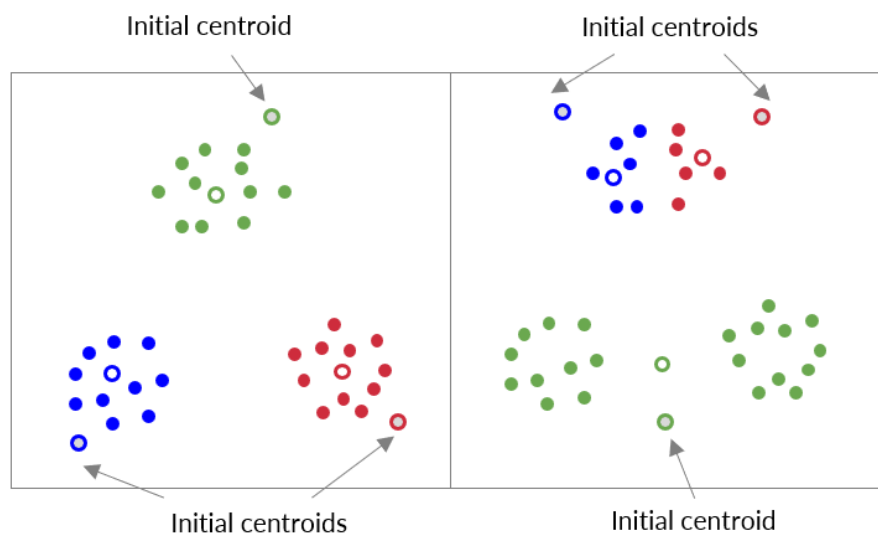
K-means is sensitive to outliers, as it relies on the mean to determine cluster centres. Even a single extreme data point can pull a centroid towards it, distorting the cluster's position and affecting the assignment of nearby points. This can lead to inaccurate or unbalanced clusters, especially when the dataset contains rare but extreme values.

To reduce this influence, it is important to detect and handle outliers before applying K-means. This can be done by removing extreme values, applying transformations such as logarithmic scaling, or by using alternative clustering models.

Initial Centroid Values

The choice of **initial centroid values** can significantly affect the outcome of K-means clustering. Since the algorithm starts with randomly selected centroids, different initialisations can lead to different final clusters, sometimes getting stuck in local minima rather than finding the best overall solution. This makes the results unstable if the process is repeated multiple times.

Consider the following as an example of how the outcome of K-means clustering can differ depending on the initialisation of centroids.



To address this, methods like K-means++ are used to improve initialisation. K-means++ selects initial centroids in a way that spreads them out across the data space, reducing the likelihood of poor starting positions. Running the algorithm several times with different seeds and comparing results is also a good practice to ensure stability and consistency in clustering outcomes. Scikit-learn's [*KMeans\(\)*](#) function uses K-means++ as its default method for initialisation.

Stopping Criteria

The stopping criteria in K-Means determine when the algorithm should end its iterative process. Typically, it stops when the centroids no longer move significantly between iterations, meaning the clusters have stabilised.

Another common condition is when the change in the WCSS falls below a defined threshold, indicating minimal improvement in cluster compactness.

Alternatively, a maximum number of iterations can be set to prevent the algorithm from running indefinitely, especially with large or complex datasets.

Combining these criteria ensures that K-Means stops efficiently, balancing computational effort with the need for stable and meaningful clustering results.

Dimensionality

As the dimensionality of data increases, K-means becomes more challenging to apply effectively. In high-dimensional spaces, data points tend to become **sparse**, meaning they are far apart from one another. This reduces the contrast between distances, making it harder for the algorithm to distinguish meaningful clusters.

Higher dimensions also lead to **greater computational costs** and make results more difficult to interpret, as visualising or explaining clusters becomes complex. To address this, dimensionality reduction techniques are often used before clustering, helping to simplify the data while retaining its key patterns.

Presence of Categorical Variables

K-means is designed for numerical data, as it relies on calculating means and distances such as the Euclidean distance.

When categorical variables are included, these operations lose meaning because categories cannot be averaged or meaningfully compared through standard distance metrics. For instance, consider a category with the following levels: “Android”, “Apple”, and “Other”.

If we encode “Android”, “Apple”, and “Other” as 0, 1, and 2, respectively, the mean of this category cannot reasonably be interpreted as “Apple” (value 1).

Similarly, if we apply one-hot encoding — $[1, 0, 0]$, $[0, 1, 0]$, $[0, 0, 1]$ — the mean would be $[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$, which does not correspond to any valid category.

Moreover, Euclidean distance assumes continuous numerical space, so it cannot properly capture relationships between categories.

Because of this, categorical variables should be handled using alternative approaches such as K-modes or K-prototypes, which replace means with modes and adapt the distance metric to work meaningfully with categorical data.

Other Forms of Clustering

K-Means is efficient and simple but has clear limitations. It assumes numerical data, spherical clusters, and similar cluster sizes. It also struggles with categorical features, irregular shapes, and outliers, which can distort the results.

K-modes clustering addresses this by working with categorical data. Instead of computing a mean, it identifies the most frequent value (mode) in each cluster, making it suitable for non-numeric features such as colours, brands, or categories.

K-prototypes clustering combines the logic of K-means and K-modes to handle mixed datasets that contain both numerical and categorical variables, ensuring meaningful clustering across different data types.

Hierarchical clustering takes a different approach by gradually merging or splitting clusters, producing a tree-like structure that allows exploration of data at various levels of detail.

Density-based spatial clustering of applications with noise (DBSCAN) groups data points based on density rather than distance from a centroid, enabling it to find clusters of irregular shapes and identify noise or outliers effectively.

Fuzzy clustering allows data points to belong to multiple clusters by assigning degrees of membership, which is useful in cases where boundaries between groups are not clearly defined.

Beyond these, other techniques such as **gaussian mixture models (GMM)** and **spectral clustering** offer more flexibility for complex data structures.

Together, these methods show that clustering is a broad and adaptable field, where choosing the right technique depends on the data type, distribution, and the insights one aims to uncover.

Applications of Clustering

Clustering has wide-ranging applications across industries because it helps uncover natural groupings within data without requiring predefined labels. In **marketing and customer analytics**, it is commonly used for **customer segmentation**, where buyers are grouped based on behaviour, demographics, or spending patterns. This allows businesses to tailor marketing strategies, personalise offers, and improve customer retention through targeted engagement.

In **healthcare**, clustering is applied to identify patterns in patient data, such as grouping individuals with similar symptoms, genetic markers, or treatment responses. This aids in disease diagnosis, drug discovery, and the development of personalised treatment plans. It also helps detect anomalies in medical imaging or patient monitoring data, supporting early intervention.

In technology and research, clustering is used in areas such as **image recognition**, **document classification**, and **anomaly detection**. For example, it helps group similar images in large datasets, cluster research papers by topic, or identify unusual network activity in cybersecurity. Across domains, clustering provides structure to complex data, revealing hidden relationships that support better decision-making and deeper insight.

Python Implementation of K-Means Clustering

Objective

The objective of this practical implementation is to apply the K-Means clustering algorithm to a customer dataset and group customers according to similarities in their annual income and spending behavior.

In this implementation, we will:

- Load and understand the dataset
- Select relevant numerical features
- Visualise the original data
- Scale the selected features
- Use the Elbow Method to identify a suitable value of K
- Train the K-Means model
- Generate cluster labels
- Examine the cluster centroids

- Visualise the final clusters
- Evaluate cluster quality using the Silhouette Score
- Interpret the discovered customer segments

Business Problem

A retail company has customer information but does not have predefined customer categories. The company wants to identify groups of customers with similar financial and spending behavior.

The company can use these customer groups to:

- Design targeted marketing campaigns
- Create personalised offers
- Identify high-value customers
- Improve customer engagement
- Develop customer-retention strategies
- Allocate marketing resources more effectively

IMPORT THE REQUIRED LIBRARIES

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import joblib
```

FEATURE SCALING

```
# Initialise StandardScaler
scaler = StandardScaler()

# Fit and transform the selected features
X_scaled = scaler.fit_transform(X)

# Convert scaled values into a DataFrame
X_scaled_df = pd.DataFrame(
```

```
        X_scaled,  
        columns=X.columns  
    )  
  
# Display the first 5 scaled rows  
X_scaled_df.head()
```

CHOOSE K USING THE ELBOW METHOD

```
wcss = []  
  
# Train K-Means models for K values from 1 to 10  
for k in range(1, 11):  
  
    kmeans = KMeans(  
        n_clusters=k,  
        init="k-means++",  
        n_init=10,  
        random_state=42  
    )  
  
    kmeans.fit(X_scaled)  
  
    # Store inertia/WCSS  
    wcss.append(kmeans.inertia_)  
  
# Plot the Elbow Curve  
plt.figure(figsize=(8, 5))  
  
plt.plot(  
    range(1, 11),  
    wcss,  
    marker="o",  
    color="purple"  
)  
  
plt.title("Elbow Method for Selecting K")  
plt.xlabel("Number of Clusters (K)")  
plt.ylabel("WCSS / Inertia")  
plt.xticks(range(1, 11))  
plt.grid(True, linestyle="--", alpha=0.4)  
plt.show()
```

TRAIN THE K-MEANS MODEL

```
# Create the K-Means model
kmeans = KMeans(
    n_clusters=5,
    init="k-means++",
    n_init=10,
    random_state=42
)

# Train the model
kmeans.fit(X_scaled)

# Generate cluster labels
labels = kmeans.labels_

# Add cluster labels to the original dataset
df["Cluster"] = labels

# Display the first 5 rows
df[
    [
        "CustomerID",
        "Gender",
        "Age",
        "Annual Income (k$)",
        "Spending Score (1-100)",
        "Cluster"
    ]
].head()
```

VIEW CLUSTER CENTROIDS

```
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
centroid_df = pd.DataFrame(
    centroids,
    columns=["Annual Income (k$)", "Spending Score (1-100)"]
).round(2)
centroid_df.index.name = "Cluster"
centroid_df
```

VISUALISE FINAL CLUSTERS

```

colors = ["red", "blue", "green", "orange", "purple"]
plt.figure(figsize=(9, 6))
for i in range(5):
    cluster_data = df[df["Cluster"] == i]
    plt.scatter(
        cluster_data["Annual Income (k$)"],
        cluster_data["Spending Score (1-100)"],
        color=colors[i],
        label=f"Cluster {i}"
    )

# Plot centroids
plt.scatter(
    centroid_df["Annual Income (k$)"],
    centroid_df["Spending Score (1-100)"],
    color="black",
    marker="X",
    s=200,
    label="Centroids"
)
plt.title("K-Means Clustering (K = 5)")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

CREATE CLUSTER PROFILES

```

cluster_profile = (
    df.groupby("Cluster")
    [
        [
            "Annual Income (k$)",
            "Spending Score (1-100)"
        ]
    ]
    .mean()
    .round(2)
)

```

ADD CUSTOMER SEGMENT NAMES

```
segment_map = {  
    0: "High Value Customers",  
    1: "Active Spenders",  
    2: "Low Engagement Customers",  
    3: "High Potential Customers",  
    4: "Average Customers"  
}  
  
df["Segment"] = (  
    df["Cluster"]  
    .map(segment_map)  
)  
  
df.head()
```

Hierarchical Clustering

Hierarchical Clustering is an unsupervised machine learning algorithm that organises data points into a hierarchy of clusters based on their similarity or distance.

The algorithm either:

- Starts with individual data points and gradually **merges** similar clusters, or
- Starts with one large cluster and gradually **divides** it into smaller clusters.

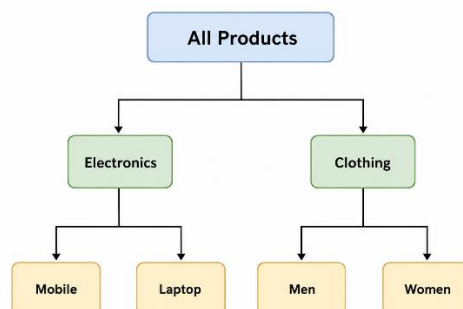
The complete clustering process creates a tree-like structure called a **dendrogram**. A dendrogram shows how observations or clusters are connected at different distance levels.

Unlike K-Means, Hierarchical Clustering does not always require the number of clusters to be specified before building the hierarchy. The desired number of clusters can be selected later by cutting the dendrogram at an appropriate height.

Why Is Hierarchical Clustering Required?

In some datasets, observations naturally exist at multiple levels.

For example, products may be organised as:



A simple clustering algorithm may identify only one final grouping. Hierarchical Clustering provides information about relationships at multiple level

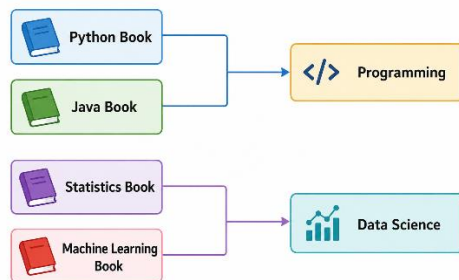
It helps answer questions such as:

- Which observations are most similar?
- Which clusters were combined first?

Real-Life Analogy: Organising Books

Imagine organising books in a library.

First, individual books are grouped into categories:



Types of Hierarchical Clustering

Hierarchical Clustering is mainly divided into:

1. **Agglomerative Hierarchical Clustering**
2. **Divisive Hierarchical Clustering**

Agglomerative Hierarchical Clustering

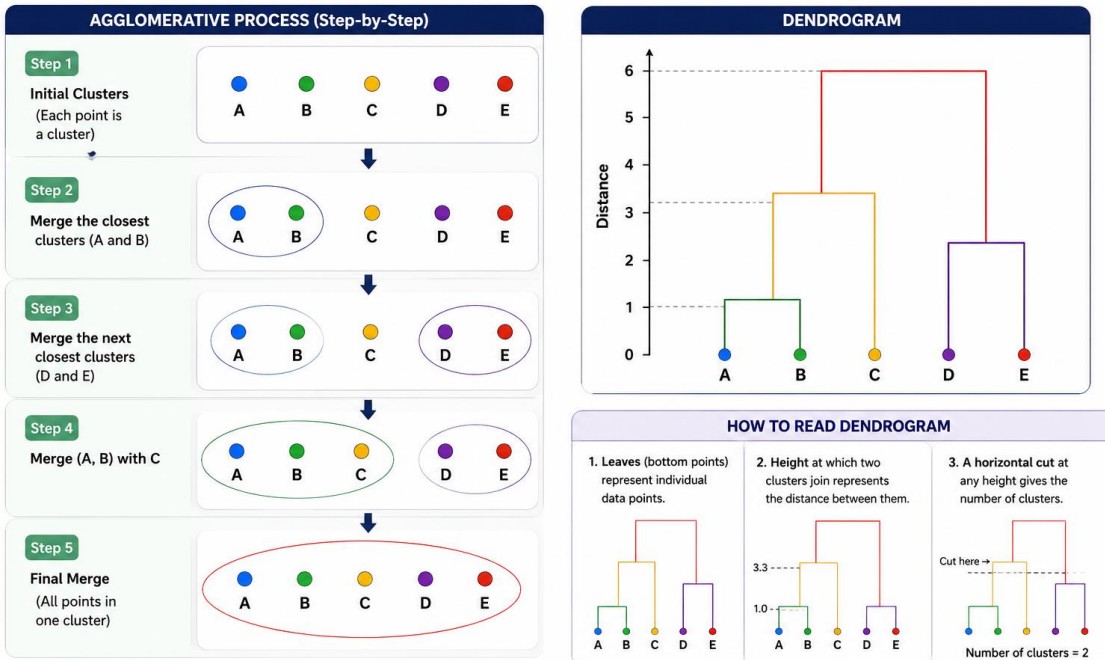
- **Agglomerative Hierarchical Clustering** is a **bottom-up clustering approach** in which every observation initially starts as a separate cluster.
- During each iteration, the two closest or most similar clusters are merged. The merging process continues until all observations form one large cluster or the required number of clusters is obtained.
- Agglomerative Clustering is the most commonly used form of Hierarchical Clustering

Working Process

Suppose the dataset contains five observations:

A, B, C, D, E

AGGLOMERATIVE HIERARCHICAL CLUSTERING (Bottom-Up Approach)



Customer-Segmentation Example

Suppose four customers have the following purchasing interests:

Customer	Primary Interest
Customer A	Smartphones
Customer B	Laptops
Customer C	T-shirts

Customer D	Jeans
------------	-------

Initially:

[A] [B] [C] [D]

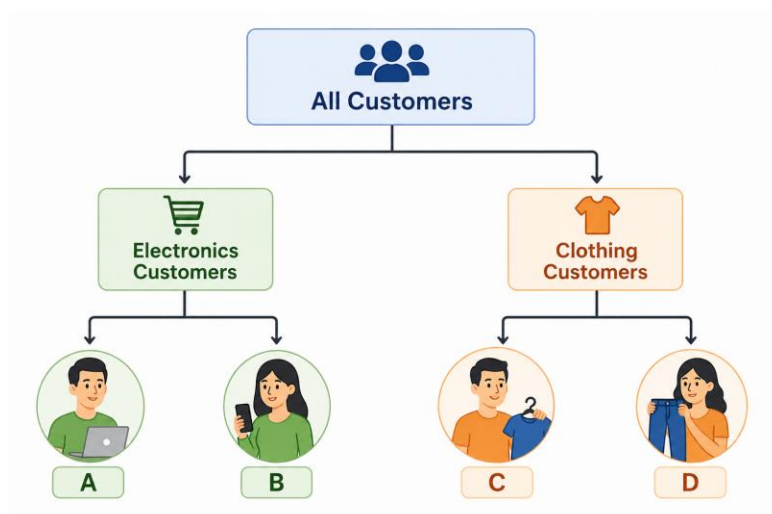
Customers A and B have similar technology-related interests:

[A, B] → Electronics Customers

Customers C and D have similar fashion-related interests:

[C, D] → Clothing Customers

The resulting hierarchy is:



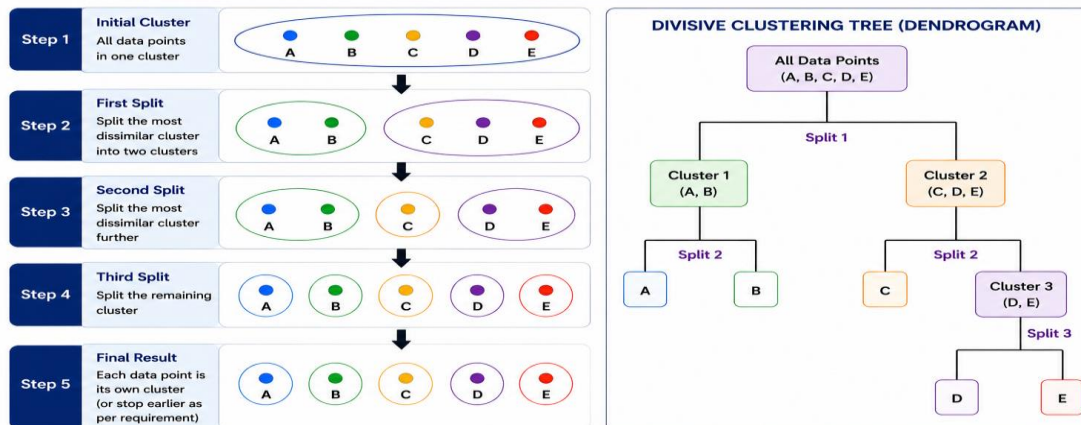
Divisive Hierarchical Clustering

Divisive Hierarchical Clustering is a **top-down clustering approach** in which all observations initially belong to one large cluster.

During each iteration, a cluster is divided into smaller groups based on the dissimilarity among its observations. The splitting process continues until every observation forms an individual cluster or the required number of clusters is obtained.

DIVISIVE HIERARCHICAL CLUSTERING (Top-Down Approach)

Divisive clustering starts with all data points in one cluster.



Agglomerative vs Divisive Clustering:-

Feature	Agglomerative	Divisive
Approach	Bottom-up	Top-down
Initial condition	Every point starts as an individual cluster	All points start in one cluster
Main operation	Merging	Splitting
Direction	Small clusters → Large cluster	Large cluster → Small clusters
Common usage	More commonly used	Less commonly used
Implementation	Generally simpler	Often more computationally demanding
Final structure	Hierarchy of merged clusters	Hierarchy of divided clusters

Dendrogram

A **dendrogram** is a tree-like diagram that represents the hierarchical relationships among observations or clusters. It shows:

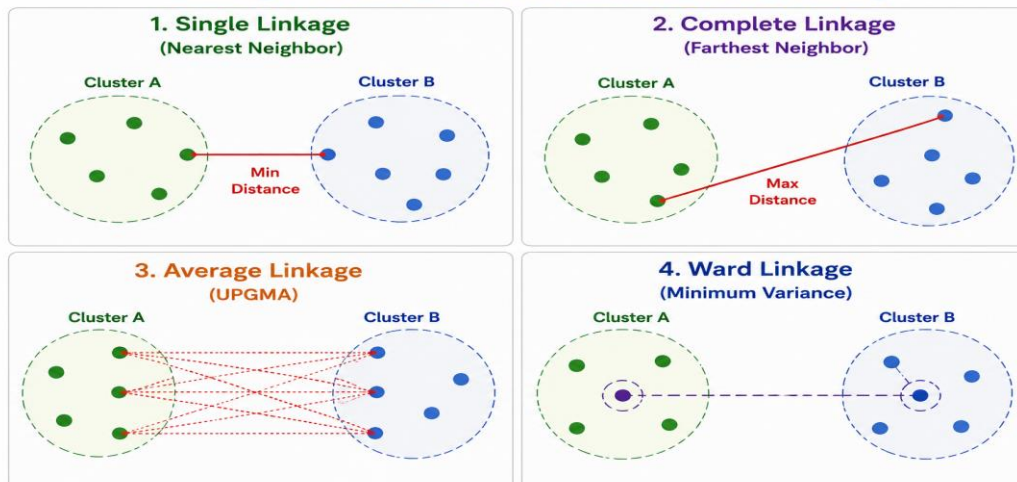
- Which observations are merged
- The order in which clusters are merged
- The distance at which clusters are combined
- The possible number of clusters

The vertical height of a merge generally represents the distance or dissimilarity between the connected clusters.

Linkage Methods in Hierarchical Clustering

Linkage Method	Meaning
Single Linkage	Uses the minimum distance between the closest points of two clusters.
Complete Linkage	Uses the maximum distance between the farthest points of two clusters.
Average Linkage	Uses the average of all pairwise distances between two clusters.
Ward Linkage	Merges the clusters that produce the smallest increase in within-cluster variance.

Linkage Methods in Hierarchical Clustering



Advantages of Hierarchical Clustering

- Does not require the number of clusters to be specified before building the hierarchy.
- Produces a dendrogram that visually represents relationships among data points and clusters.
- Supports different distance metrics and linkage methods for flexible cluster formation.

Limitations of Hierarchical Clustering

- Computationally expensive for very large datasets.
- Sensitive to noise, outliers, feature scaling, and the selected linkage method.
- Once clusters are merged or split, the decision generally cannot be reversed.

Fuzzy Clustering

Fuzzy Clustering is a soft-clustering technique in which one observation can belong to multiple clusters with different membership values.

Traditional clustering methods generally assign each observation to only one cluster. Fuzzy Clustering allows partial membership because real-world groups may overlap.

Each membership value lies between:

$$0 \leq u_{ij} \leq 1$$

where:

- u_{ij} = Membership value of observation i in j cluster
- 0 = No membership
- 1 = Complete membership
- A value between 0 and 1 = Partial membership

Why is Fuzzy Clustering Required?

Real-world categories do not always have clear boundaries.

For example, a customer may purchase:

- Electronics
- Clothing
- Sports products

Assigning the customer completely to only one category may remove useful information.

Fuzzy Clustering represents the customer's relationship with multiple groups using membership value

Real-Life Example of Fuzzy Clustering

Suppose Customer A purchases both electronics and clothing.

Fuzzy Clustering produces:

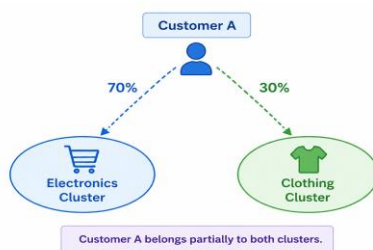
Cluster	Membership Value
Electronics	0.70
Clothing	0.30

Interpretation:

- Customer A has **70% membership** in the Electronics cluster.
- Customer A has **30% membership** in the Clothing cluster.
- The total membership is:

$$0.70 + 0.30 = 1$$

The customer belongs more strongly to the Electronics cluster but also has a meaningful relationship with the Clothing cluster



Advantages

- Supports overlapping clusters.
- Handles uncertainty effectively.
- Provides detailed membership information.

Disadvantages

- Requires the number of clusters in advance.
- Sensitive to noise and outliers.
- Has a higher computational cost.

Limitations of Fuzzy Clustering

- Requires the number of clusters to be specified in advance.
- Sensitive to noise, outliers, and feature scaling.
- Has higher computational complexity due to membership calculations.

Hard Clustering

Hard Clustering, also known as **Crisp Clustering**, is a clustering approach in which each data point is assigned completely to **only one cluster**.

A data point either belongs to a cluster or does not belong to it. Therefore, cluster membership is represented using binary values:

$$u_{ij} \in 0,1$$

Where:

- u_{ij} = Membership of data point i in cluster j
- 1 = The data point belongs to the cluster
- 0 = The data point does not belong to the cluster

Each data point has complete membership in one cluster and zero membership in all other clusters.

Example

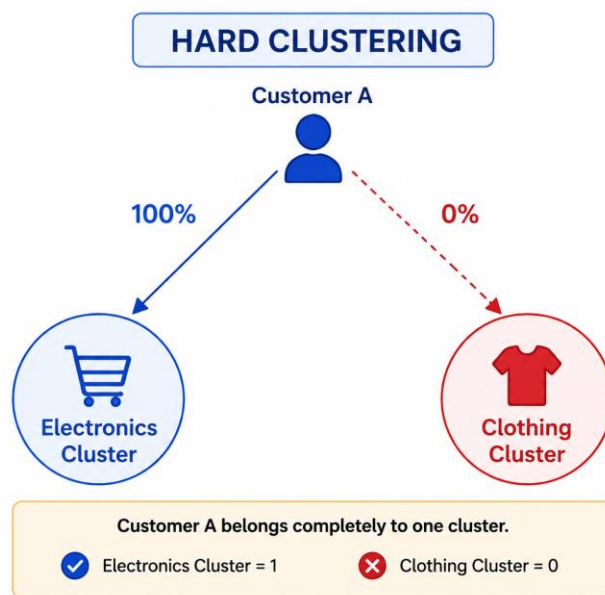
Suppose Customer A mainly purchases electronic products.

Hard Clustering may assign Customer A as:

Cluster	Membership Value
Electronics Cluster	1 or 100%
Clothing Cluster	0 or 0%

Therefore:

$$1+0=1$$



Limitation

Hard Clustering may not represent overlapping or uncertain data effectively.

For example, if a customer purchases both electronics and clothing equally, Hard Clustering still assigns the customer completely to only one cluster. This may ignore useful information about the customer's interest in the other category.

In such situations, **Fuzzy Clustering** may be more suitable because it allows partial membership in multiple clusters.

Hard Clustering vs Fuzzy Clustering

Feature	Hierarchical Clustering	Fuzzy Clustering
Main concept	Creates a hierarchy of clusters	Calculates degrees of cluster membership
Structure	Tree-like	Overlapping clusters
Main output	Dendrogram	Membership matrix and cluster centers

Cluster assignment	Usually one cluster at a selected hierarchy level	Partial membership in multiple clusters
Membership values	Not normally used	Values between 0 and 1
Common algorithm	Agglomerative Clustering	Fuzzy C-Means
Number of clusters	Can be selected using a dendrogram	Usually specified before training
Best use case	Discovering nested relationships	Handling overlapping cluster boundaries
Example	Product-category hierarchy	Customer interests across multiple categories

DISCLAIMER: This learning course may contain images, both still and moving, including photographs, advertisements, third-party trademarks, video clips etc. that may constitute

copyrighted material procured from open-source, public domain, social media, or other print or digital publications, with the sole and bona fide intention of imparting education, disseminating information and illustrating an academic concept or an issue. We believe that this constitutes a 'fair dealing' exception under Section 52 of the Indian Copyright Act, 1957. Throughout this course, you may see examples of different brands and institutions. These have been chosen from specific cases in a way that would aid in demonstrating concepts. These do not reflect the current performance of any company. Any opinion expressed is strictly for educational purposes.